Performance Testing Strategy

The purpose of performance testing

Performance testing is the type of testing performed to determine the performance of the system to measure, validate or verify quality attributes of the system like responsiveness, speed, scalability, stability under variety of load conditions.

Some goals/questions to ask

- Ensure SLA is upheld. If we are promising no slower than 3 seconds to load a page, we must ensure we are within that threshold at the anticipated load
- Ensure the application is functioning the same under load as it is when closer to idle. Checking for errors in responses is critical.
- Does the application performance degrade over a longer test? Or is it consistent?
- How do we feel if we increased our customer base by 50%? Can the application handle that anticipated load?

Types of Performance Testing

Below is a high level description of the types of performance tests you may perform. This isn't an exhaustive list, just the common types of tests you may perform while testing our applications. Blazemeter has wonderful documentation, if you are interesting in learning more about the types of performance tests, visit this link https://www.blazemeter.com/blog/performance-testing-vs-load-testing-vs-stress-testing

Load Testing

Load testing is the most common type of performance test we run here at Ascensus. The goal of load testing is to measure the response times of the application at our anticipated level of throughput (req/sec). The goal is not to find the breaking point, rather it is to ensure satisfactory response times given a typical load. Throughput is important to learn about in performance testing as it is the standard way to measure whether performance is maintained or degrading as the test is performed. For instance, if we find in New Relic that our typical load is 10 req/sec, but in our test we saw 20 req/sec with no reduction in response times and no errors returns, we are confident in the application's performance. See this link here https://testguild.com/performance-testing-what-is-throughput/

Baseline Regression Testing

Baseline regression testing is a repeatable test at a constant load used to ensure we have not degraded performance in an obvious way prior to releasing to production. A use case for this is setting up your production load test in a lower environment, such as QA, and running a 10 minute test after every deployment. Any test that breaks the failure thresholds will fail in the pipeline and notify the team. This is useful for catching any obvious issues before we release to production

Soak Testing (endurance)

Soak testing, or endurance testing, is a type performance test that is used to detect memory leaks or performance degradation over a long period of time. The test will typically be of lower load than typical, but at a much longer duration. An example would be if we load tested the application using 10 concurrent users over 30 minutes, a soak test may be 2-4 concurrent users over 10 hours. Ensure your team has a way to measure CPU/Memory over time in the application. This is a very valuable type of test and should always be used when new applications are being provisioned.

Stress Testing

Stress testing is similar to load testing, except we will slowly ramp up load until the application stops functioning at an acceptable level. It is NOT necessary to completely bring down the servers to find a breaking point, we can simply look for the response times to significantly increase and we know we are close to the true breaking point.

Spike Testing

Spike testing is the next level up from stress testing. With this type of test, we are increasing load until the application no longer responds, then shutting down the test to see if the application can recover on it's own. This isn't a common test here at Ascensus because of the high risk nature of performing this type of test. If it were utilized, it should be in a controlled environment that is isolated and won't affect active users.

Scalability Testing

The purpose of scalability testing is to see how adding resources to our application affects the performance. A way to test this would be to perform a load test, make changes to the application (whether increasing memory on a server, adding more servers, adding caching, etc), then perform the load test again and measure for any performance increases or decreases. This type of test is usually performed in tandem with a DevOps engineer.

How To Test

In the majority of cases, we will be testing some type of web application whether we are simulating a user navigating our app using a browser or we are testing an API directly. For this, we use Taurus and Blazemeter. To learn more about Taurus with blazemeter, click here Blazemeter With Taurus

Browser/UI Testing

Using blazemeter/taurus/jmeter

The intention of this test is to make requests to the server as close as possible to how an end user would be making requests using their browser. This involves making the request to the endpoint, storing any required cookies, and executing any AJAX calls found on the page. This typically does not involve any client side javascript code execution.

Using selenium/playwright

It is possible to go one level higher and use an actual browser to performance test a website by using selenium or playwright. However, we have not tested these tools yet and cannot recommend them. More research has to be done on these tools. I predict they will require a lot more server resources to apply a given load and will be flakier overall than simply using taurus. The advantage would be less manual modeling of each HTTP request as these tools utilize an actual browser which will do most of the work for you.

API Testing

This is the simplest type of performance test we will perform as API's are typically just a collection of endpoints. Typically there won't be any complex http requests here, just simple get or post requests to a given endpoint. For this, we will definitely leverage the taurus/blazemeter pattern

Batch Testing

Testing a batch process is more ad hoc in nature, as taurus doesn't really apply here since we are not testing an http endpoint. We are simply ensuring the batch process completes in a satisfactory amount of time which is defined by the team.

Where to Test

Choosing the right environment to test in is critical to acquiring useful results. Choosing the wrong type of test in a given environment can lead to misleading results.

Production like environment or production itself

Production (or equivalent) is the best environment to test in for the most accurate results. Our production environments are typically much more powerful than our lower environments due to cost. Not only are they more powerful, the app level configurations are typically different. You are getting true results when testing against production or production like environments. With that said, there is inherit risk in testing in a production environment, so it should be a last resort after other alternatives are explored.

A great alternative to production is boosting the capabilities of a lower environment. An example of this would be pointing a QA database to another, more powerful database.

The process for testing in production

Testing in production can be risky if the appropriate steps are not taken.

- The team communicates ahead of time that they are intending to perform a test in a production environment. This requires creating a CAB ticket and attending the CAB meeting to discuss the intention of the test.
- Prior to and upon completion of the test, an email must be set to maintenance@ascensus.com informing the group a performance test is being run and which applications are affected
- A DBA and DevOps engineer are accompanying the tester
- The test is performed off hours

Non production environments

Non production environments can also be useful to test in, but you must consider the potential hardware limitations which may lead to misleading results. These type of environments are best used for these types of tests:

- Testing your test script to ensure it functions properly
- Regression testing at a fixed load
- Spike testing (to bring down the servers and see if the application can recover)
- Soak testing (for memory leaks or performance degradation)

These types of environments should not be used to determine actual performance of the system

When to Test

- Completely new service or application
- Automated regression of previously tested service or application
- When there are changes to the underlying service or application which may affect performance
- When an influx of new traffic will be coming. Example, new business is coming on board and we expect our user base to grow by 50%.

Reporting

- Use Blazemeter for results
- Use graphical results to share with team. We have used confluence pages in the past with screenshots and links to each report.